

Теория для защиты — Ecosystem Simulation

Yermek Aubayev (551098)

2026-09-14

Перевод `theory.md` на русский. Компаньон — [defense_script_ru.md](#) (репетиция/Q&A/живое кодирование на русском) или [defense_script.md](#) (английский оригинал). Код, номера строк и ссылки на файлы оставлены как есть — это факты, не текст для перевода. Каждая строка ниже сверена с актуальным исходным кодом на 2026-09-14 — если код изменится после этой даты, перепроверь номер строки перед тем, как называть его профессору.

Терминология совпадает с `docs/report/final_report.md` намеренно. На самой защите не переключайся с одного слова на другое для одного и того же понятия (например, не говори “event queue” в одном предложении и “task list” в следующем) — отчёт и устный ответ должны звучать так, будто их писал и понимал один и тот же человек. Поскольку сама защита, скорее всего, идёт на английском, привязывай русские термины ниже к английским оригиналам в скобках — это те слова, которые реально нужно произносить вслух.

Часть А — Принципы ООП

А.1 Инкапсуляция (Encapsulation)

Определение. Объединение данных объекта с операциями, которые с ними работают, и контроль над тем, как к этим данным можно обратиться снаружи — единственная дверь внутрь — это собственные методы объекта.

Код. [World.java:85](#) — `getEntities()` возвращает `new ArrayList<>(this.entities)`, защитную копию, а не сам живой список.

Сказать (по-английски). “A `private` field only stops you from *replacing* World’s reference. Without the defensive copy, a caller could still get the live list through the getter and mutate it — add or remove entities behind World’s back. The copy is what actually closes that door.” **Смысл:** поле `private` само по себе не даёт только *заменить* ссылку `World`. Без защитной копии вызывающий код всё равно мог бы получить живой список через геттер и мутировать его — добавлять или удалять сущности в обход `World`. Именно копия реально закрывает эту дверь.

⚠ **Уже поднималось.** Профессор указал, что ранняя версия этого раздела смешивала инкапсуляцию с сокрытием информации. Держи их наглядно отдельно — см. А.2.

А.2 Соккрытие информации (Information Hiding)

Определение. Дисциплина выбора, для каждого атрибута, ровно той видимости, области действия и изменяемости, которая ему нужна, и не больше — чтобы никакой код снаружи объявляющего класса не мог начать зависеть от деталей, которые могут позже измениться. Проект применяет 3-шаговый метод к каждому полю: (1) всегда `private`, (2) уровень экземпляра или уровень класса, (3) константа или переменная.

Применение 3 шагов к Entity / Organism

Атрибут	1. Private?	2. Область	3. Константа или переменная
Entity.id	Да	Экземпляр	Константа — <code>final</code> , присваивается один раз в конструкторе, никогда не переприсваивается
Entity.nextId	Да	Класс (<code>static</code>)	Переменная — увеличивается при каждом <code>new Entity(...)</code>
Organism.energy	Да	Экземпляр	Переменная — меняется через <code>gainEnergy / consumeEnergy</code>

Код. [Entity.java:24](#) (`private final int id`) и [Entity.java:29](#) (`private static int nextId = 1`).

Сказать (по-английски). “Every field in the `Entity` → `Organism` → `Animal` → `Predator/Herbivore` chain is `private`, including fields declared in the abstract base classes — not `protected`. A subclass reaches inherited state only through the accessor, the same as any outside caller would.” **Смысл:** каждое поле в цепочке `Entity` → `Organism` → `Animal` → `Predator/Herbivore` — `private`, включая поля, объявленные в абстрактных базовых классах, — не `protected`. Подкласс достаёт унаследованное состояние только через accessor-метод, точно так же, как и любой внешний вызывающий код.

⚠ **Уже поднималось — знать твёрдо.** Профессор поймал реально `protected`-поля вопреки заявлениям про геттеры (`Entity.id/x/y/alive/...`, `Organism.energy/age/maxEnergy`, `Animal.speed/visionRange`, `Predator.attackPower`, `Herbivore.defensePower`), и поймал `Predator.hunt()`, читавший унаследованное поле `visionRange` напрямую вместо вызова `getVisionRange()`. Оба места исправлены — проверено: каждое поле выше читается как `private` в актуальном исходном коде (проверено напрямую, не по отчёту). **Инкапсуляция vs. сокрытие информации, одной фразой:** инкапсуляция — это контроль доступа к состоянию одного объекта (контракт метода); сокрытие информации — это свободна ли внутренняя реализация иерархии меняться (объявление поля). Не путать одно с другим.

A.3 Наследование (Inheritance)

Определение. Подкласс переиспользует и расширяет состояние и поведение суперкласса, выражая настоящее отношение “является” (is-a).

Код. [Organism.java:15](#) `extends Entity`, [Animal.java:16](#) `extends Organism implements Movable`, [Herbivore.java:9](#) `extends Animal implements Reproducible`.

Сказать (по-английски). “`Plant` sits directly under `Organism`, not under `Animal`, even though it’s obviously a living thing. That was a deliberate check: if `Plant` inherited from `Animal implements Movable`, code holding an `Animal` reference could legally call `move()` on something that structurally cannot move — a Liskov Substitution violation. Keeping `Movable` a separate interface (A.7) is what makes that tree honest.” **Смысл:** `Plant` стоит прямо под `Organism`, не под `Animal`, хотя

растение — очевидно живой организм. Это осознанная проверка: если бы `Plant` наследовался от `Animal implements Movable`, код, держащий ссылку на `Animal`, мог бы законно вызвать `move()` у того, что структурно не умеет двигаться, — нарушение принципа подстановки Лисков. Отдельный интерфейс `Movable` (A.7) делает это дерево честным.

A.4 Абстракция (Abstraction)

Определение. Представление только тех черт объекта, которые действительно нужны, с откладыванием вопроса *как именно* на конкретные подклассы.

Код. `Entity.java:56` `public abstract void update();` и `SimulationEvent.execute(SchedulingContext) / getDescription()`.

Сказать (по-английски). “The base class can’t know the concrete behavior in advance — a `Plant` grows, a `Predator` hunts, a `DeathEvent` records a death regardless of cause. One abstract method per concern is what lets the engine stay completely ignorant of concrete types (A.5.1).” **Смысл:** базовый класс не может знать конкретное поведение заранее — `Plant` растёт, `Predator` охотится, `DeathEvent` фиксирует смерть независимо от причины. Один абстрактный метод на концепцию — это то, что позволяет движку оставаться полностью не осведомлённым о конкретных типах (A.5.1).

A.5 Полиморфизм — четыре разные формы

A.5.1 Полиморфизм включения (время выполнения / динамическая диспетчеризация)

Код. `SimulationEngine.java:132` — `event.execute(this)`, диспетчеризуется во время выполнения на то, какой именно из шести подклассов событий был извлечён.

Сказать (по-английски). “The engine never asks ‘what kind of event is this?’ It calls `execute()` through the base `SimulationEvent` type and the JVM’s dynamic dispatch selects the override. Same mechanism one level down: `EntityActivityEvent` calls `entity.update()` through the base `Entity` type.” **Смысл:** движок никогда не спрашивает “что это за тип события?”. Он вызывает `execute()` через базовый тип `SimulationEvent`, а динамическая диспетчеризация JVM сама выбирает переопределённый метод. Тот же механизм уровнем ниже: `EntityActivityEvent` вызывает `entity.update()` через базовый тип `Entity`.

Честный контрпример, стоит сказать самому. Узлы AST для правил (`ConditionNode`, `NumericNode`, `RuleAction` — см. Часть C) обрабатываются в `Evaluator` через исчерпывающий `switch` (`Evaluator.java:37`), **не** через виртуальную диспетчеризацию — метода `evaluate()` на каждом узле нет. Это осознанная, современная альтернатива (проверяемая компилятором исчерпываемость над закрытым набором узлов), а не пример полиморфизма включения. Умение провести эту границу самому, не дожидаясь вопроса, — как раз та точность, которая хорошо звучит на устном экзамене.

A.5.2 Полиморфизм перегрузки (время компиляции)

Код. `Statistics.java:85` `recordBirth(String)` и `Statistics.java:100` `recordBirth(String, int);` у `RuleParseException.java` четыре конструктора (строки 31/43/58/70), все называются `RuleParseException`, но различаются списком параметров.

Сказать (по-английски). “The compiler picks the matching signature at **compile time**, from the argument types at the call site — the opposite resolution moment from `update()` / `execute()` in A.5.1, which resolve at **runtime** from the object’s actual class. That compile-time-vs-runtime distinction is the single most common follow-up question here.” **Смысл:** компилятор выбирает подходящую сигнатуру во **время компиляции**, по типам аргументов в месте вызова — противоположный момент разрешения по сравнению с `update()` / `execute()` из A.5.1, которые разрешаются **во время выполнения**, по фактическому классу объекта. Это различие (компиляция vs. выполнение) — самый частый уточняющий вопрос здесь.

A.5.3 Параметрический полиморфизм (обобщения / generics)

Код. [EventQueue.java:28](#) — `class EventQueue<T extends SimulationEvent> .`

Сказать (по-английски). “The bound does real compile-time work: the compiler rejects enqueueing anything that isn’t a `SimulationEvent`, while `EventQueue` itself never needs to know which concrete subtype it holds. Only `EventQueue<SimulationEvent>` is actually instantiated in this project, but the bound isn’t decorative — a future, narrower event queue would work with this class completely unmodified.” **Смысл:** ограничение типа реально работает уже на этапе компиляции: компилятор отклонит попытку положить в очередь что угодно, кроме `SimulationEvent`, при этом самому `EventQueue` никогда не нужно знать, какой именно конкретный подтип он хранит. В проекте реально создаётся только `EventQueue<SimulationEvent>`, но ограничение не декоративное — будущая, более узкая очередь событий заработала бы с этим классом вообще без изменений.

A.5.4 Полиморфизм приведения (Coercion)

Код. [Predator.java:170](#) — `(double) this.attackPower / (this.attackPower + herbivore.getDefensePower()) .`

Сказать (по-английски). “Both operands start as `int`. The explicit cast on the left operand forces Java’s binary numeric promotion rule to widen the *other*, uncast `int` operand to `double` too, before the division runs — that implicit widening of the second operand is the coercion. This is deliberately **not** illustrated with `Integer.parseInt(...)` — that’s an explicit method call performing a conversion, not a language-level coercion. Conflating the two is a labeling mistake worth avoiding out loud.”

Смысл: оба операнда начинаются как `int`. Явное приведение типа у левого операнда заставляет правило бинарного числового расширения Java расширить и *другой*, неприведённый `int`-операнд до `double`, прежде чем выполнится деление — это неявное расширение второго операнда и есть приведение (coercion). Это намеренно **не** иллюстрируется через `Integer.parseInt(...)` — это явный вызов метода, выполняющий преобразование, а не приведение на уровне языка. Смешивать эти два понятия — ошибка терминологии, которую стоит не допустить вслух.

A.6 Композиция (Composition)

Определение. Отношение “имеет” (has-a): объект хранит ссылки на другие, независимо созданные объекты и использует их публичное поведение.

Код. [SimulationEngine.java:36-42](#) — шесть `final` полей (`World`, `RuleVocabulary`, `RuleEngine`, `Statistics`, `EntityFactory`, `EventQueue<SimulationEvent>`), все создаются внутри собственного конструктора `SimulationEngine`.

Сказать (по-английски). “None of these composed objects *is a kind of* `SimulationEngine`, and none meaningfully outlives one running simulation — so composition, not inheritance, is the structurally correct relationship. It also keeps the engine’s internals encapsulated: nothing outside can reach `RuleEngine` except through the narrow `SchedulingContext` seam the engine itself implements.” **Смысл:** ни один из этих составных объектов *не является видом* `SimulationEngine`, и ни один из них осмысленно не переживает одну работающую симуляцию — так что композиция, а не наследование, структурно правильное отношение здесь. Это также сохраняет внутренности движка инкапсулированными: ничто снаружи не может добраться до `RuleEngine`, кроме как через узкий интерфейс `SchedulingContext`, который сам движок и реализует.

A.7 Подтипизация / мультитипизация (через интерфейсы)

Определение. Один объект одновременно удовлетворяет нескольким независимым типам.

Код. `Predator` — это `Animal` (наследование), `Movable` (унаследовано транзитивно через `Animal`) и `Reproducible` (объявлено напрямую) — три типа, один объект. Доказательство — в [Main.java:67](#) (`demonstrateMultityping()`), вызывается из `main()` в [Main.java:23](#) и в [RuleVocabulary.java:244-246](#), где команды `move` / `reproduce` регистрируются на **интерфейс**, а не на список конкретных классов.

Сказать (по-английски). “`Main.demonstrateMultityping()` builds exactly one `Predator` and assigns that same reference to five variables of five different declared types — `Entity`, `Organism`, `Animal`, `Movable`, `Reproducible` — then prints `entityRef == animalRef`, which is `true`. That’s the proof: reference equality compares the underlying object, not the compile-time type of the variable holding it.” **Смысл:** `Main.demonstrateMultityping()` создаёт ровно один `Predator` и присваивает ту же самую ссылку пяти переменным пяти разных объявленных типов — `Entity`, `Organism`, `Animal`, `Movable`, `Reproducible` — а затем печатает `entityRef == animalRef`, что даёт `true`. Это и есть доказательство: сравнение ссылок сравнивает сам объект, а не тип переменной, известный во время компиляции.

⚠ **Уже поднималось.** Профессор попросил именно такое доказательство из клиентской программы; раньше его не было. `Main.demonstrateMultityping()` написан специально в ответ на это и выполняется в самом начале каждого запуска программы, до открытия GUI — будь готов реально запустить программу и показать вывод в консоли вживую, если попросят.

A.8 Обработка исключений (Exception Handling)

Определение. Код, который может завершиться неудачей, сигнализирует об этом через чётко определённый тип исключения, а вызывающий код, который этого ожидает, **ловит исключение и предпринимает реальное восстанавливающее действие** — а не просто определяет класс исключения и даёт ему распространиться дальше, и не просто ловит его, чтобы напечатать сообщение и продолжить как ни в чём не бывало.

Код. Проверяемое: [RuleParseException.java:13](#) extends `Exception`. Непроверяемое: [SimulationException.java:32](#) extends `RuntimeException`. Реальный catch + восстановление #1: [RuleEditorPanel.java:119-127](#) `doReload()`. Реальный catch + восстановление #2: [MainWindow.java:61-69](#) и такой же блок на строке 185.

Сказать (по-английски). “RuleParseException is checked because callers are *expected* to catch it and recover — a malformed rule is ordinary user input, not a bug. SimulationException is unchecked because its causes (e.g. negative world dimensions) indicate a programming error, not something to routinely expect and recover from at runtime. RuleEditorPanel.doReload() is genuine handling, not decoration: because RuleEngine.loadRules() parses into a staging list and only replaces the active rules on full success, catching RuleParseException here leaves the engine’s active rules **exactly as they were** — verified by RuleRepositoryTest.reloadingCorruptedFile_preservesLastValidRules, not just asserted in prose.” **Смысл:** RuleParseException — проверяемое, потому что вызывающий код *обязан* его поймать и восстановиться — некорректное правило это обычный пользовательский ввод, а не баг. SimulationException — непроверяемое, потому что его причины (например, отрицательные размеры мира) указывают на ошибку программирования, а не на то, что нужно регулярно ожидать и от чего восстанавливаться во время выполнения. RuleEditorPanel.doReload() — настоящая обработка, не украшение: поскольку RuleEngine.loadRules() парсит во временный список и заменяет активные правила только при полном успехе, перехват RuleParseException здесь оставляет активные правила движка **точно такими же, какими они были** — это независимо проверяется тестом RuleRepositoryTest.reloadingCorruptedFile_preservesLastValidRules, а не просто утверждается в тексте.

⚠ **Уже поднималось.** Профессор сказал, что исходный раздел про исключения показывал только определения конструкторов, ни разу не показывая реальный catch-блок, а отдельно у MainWindow был по-настоящему слабый catch: он только писал в System.err — невидимо в упакованном Swing-приложении. Оба места в MainWindow теперь пишут в экранный EventLogPanel. Если спросят, что вообще требуется от “обработки”, помимо самого факта наличия try/catch — сказать: она должна (1) остановить распространение исключения в падение программы, (2) оставить программу в одном конкретном, чётко определённом состоянии, (3) сообщить пользователю, что произошло. Catch-блок, который только печатает в консоль, не удовлетворяет ни второму, ни третьему условию.

A.9 Расширяемость (принцип открытости/закрытости, Open/Closed)

Определение. Новую функциональность можно добавить, не изменяя существующие, уже протестированные классы.

Код. RuleVocabulary.java:75-85 — registerReadable / registerWritable / registerCommand, единственная точка, через которую проходит новый атрибут или команда.

Сказать точно — не преувеличивай (по-английски). “Composing *new rule logic* from attributes/commands already registered needs zero Java changes. Exposing a genuinely *new* attribute or command needs exactly one registerX(...) call in RuleVocabulary and a recompile — nothing in Lexer, Parser, Rule, or RuleEngine changes. That boundary is stated openly in the report; don’t let a question push you into claiming the DSL is more open than that.” **Смысл:** составление *новой логики правил* из уже зарегистрированных атрибутов/команд не требует ни строчки Java. Открытие по-настоящему *нового* атрибута или команды требует ровно одного вызова registerX(...) в RuleVocabulary и перекомпиляции — ничего в Lexer, Parser, Rule или RuleEngine не меняется. Эта граница прямо заявлена в отчёте; не давай вопросу увести себя в заявление, что DSL более открыт, чем это. Более острый нюанс именно про расширяемость видов — в defense_script_ru.md §5.

A.10 Модульность (Modularity)

Определение. Программа, построенная как набор небольших независимых частей, каждая из которых отвечает за одну задачу; в ООП модулем является класс.

Код. Структура пакетов: `entities/`, `events/`, `rules/`, `simulation/`, `statistics/`, `gui/` — `entities` никогда не импортирует `gui`; `rules` знает про `entities` только через `Movable/Reproducible/Entity`, никогда через конкретный вид.

Сказать — и честно назови пробел сам, не дожидаясь, пока поймут (по-английски). “It’s not perfectly modular everywhere, and that’s stated in the report rather than hidden: the concrete species list (`Predator / Herbivore / Plant`) is spelled out independently in two places — `Statistics.adjustSpeciesCounter` ([Statistics.java:130](#)) and `EntityFactory.createOffspringNear`’s `instanceof` chain ([EntityFactory.java:78-87](#)). Removing that would mean adding a species-registry abstraction solely to delete one small `instanceof` chain — a deliberate tradeoff, not an oversight, trading a generic mechanism for something a new reader can actually follow.” **Смысл:** модульность не идеальна абсолютно везде, и это заявлено в отчёте, а не спрятано: список конкретных видов (`Predator / Herbivore / Plant`) независимо перечислен в двух местах — в `Statistics.adjustSpeciesCounter` и в цепочке `instanceof` метода `EntityFactory.createOffspringNear`. Убрать это означало бы добавить абстракцию-реестр видов исключительно ради удаления одной небольшой цепочки `instanceof` — осознанный компромисс, а не недосмотр: обменять обобщённый механизм на что-то, что новый читатель реально может проследить.

A.11 Иерархия (Hierarchy)

Определение. Организация связанных понятий по специализации — не просто “эти два понятия связаны”, а какое из них более общее, а какое более конкретное, и сколько уровней специализации реально нужно предметной области.

Код. Дерево из четырёх уровней: `Entity` → `Organism` → `Animal` → {`Predator`, `Herbivore`}, `Plant` ответвляется прямо от `Organism`.

Сказать (по-английски). “This isn’t just documentation — `RuleVocabulary.requireReadable / requireWritable / requireCommand` validate a rule by walking this exact tree with `Class.isAssignableFrom`, so a rule targeting `Organism` is automatically valid for everything below it without `RuleVocabulary` ever naming `Predator` or `Herbivore`. Get the shape wrong — e.g. `Plant` under `Animal` — and an invalid rule like `Plant | move` would pass validation, because the vocabulary trusts the hierarchy to tell the truth about what a type actually is.” **Смысл:** это не просто документация — `RuleVocabulary.requireReadable / requireWritable / requireCommand` проверяют правило, обходя именно это дерево через `Class.isAssignableFrom`, так что правило, нацеленное на `Organism`, автоматически валидно для всего, что ниже, при этом `RuleVocabulary` ни разу не называет `Predator` или `Herbivore` напрямую. Ошибись в форме дерева — например, помести `Plant` под `Animal` — и некорректное правило вроде `Plant | move` пройдет валидацию, потому что словарь доверяет иерархии говорить правду о том, чем конкретный тип реально является.

А.12 Повторное использование (Reuse)

Определение. Применение уже существующего, уже отлаженного кода для новой задачи вместо того, чтобы писать его заново. Два механизма: наследование (переиспользование *внутреннего* состояния суперкласса через его собственные accessor-методы) и композиция (переиспользование *публичного* поведения другого объекта через хранение ссылки на него).

Код. Переиспользование через наследование: `Predator` / `Herbivore` оба переиспользуют управление энергией/возрастом от `Organism` и состояние движения от `Animal`.

Переиспользование через композицию: до появления `EntityFactory` логика создания видовых значений по умолчанию дублировалась один раз для начальной популяции и ещё раз внутри собственного `reproduce()` каждого вида; изменение конфигурации в одном месте молча не доходило до другого.

Часть В — Теория дискретно-событийного моделирования (Discrete-Event Simulation)

Общее определение (учебник, не специфика проекта). Дискретно-событийная симуляция моделирует систему как последовательность мгновенных событий, каждое из которых меняет состояние системы в конкретный момент модельного времени. Время продвигается только скачком к следующему запланированному событию — никогда фиксированным шагом Δt — потому что между событиями ничего не меняется, и продвижение через эти промежутки было бы напрасным вычислением. Это противоположность **симуляции с фиксированным временным шагом** (time-slicing / fixed-timestep), которая продвигает время на постоянную Δt на каждой итерации и заново оценивает каждую сущность независимо от того, изменилось ли в ней что-то.

Базовые термины и где каждое понятие живёт в этом проекте:

Понятие DES	В этом проекте
Часы симуляции (simulation clock)	<code>SimulationEngine.clock</code> (приватный <code>int</code>)
Список будущих событий, FEL (future-event list)	<code>EventQueue<SimulationEvent></code> , на основе <code>java.util.PriorityQueue</code> — EventQueue.java:28
Событие	<code>SimulationEvent</code> и его шесть подклассов — SimulationEvent.java:36
Собственная метка времени события	<code>scheduledTime</code> , задаётся один раз при создании — SimulationEvent.java:41
Переменные состояния	поля <code>Entity</code> / <code>World</code> / <code>Environment</code> / <code>Statistics</code> — меняются только внутри чьего-то <code>execute()</code>
Управляющий цикл	<code>SimulationEngine.advanceTo(int)</code> — SimulationEngine.java:128

Фраза, которая доказывает, что это по-настоящему дискретно-событийная система, а не цикл с фиксированным шагом, надевший это название: `advanceTo()` извлекает единственное ближайшее по времени событие и устанавливает `clock =`

`event.getScheduledTime()` — принимает собственное время извлечённого события, вместо того чтобы увеличивать часы на фиксированный шаг и затем опрашивать каждую сущность. Часы никогда не продвигаются по собственной инициативе движка — они только принимают метку времени, которая уже существовала на каком-то событии.

Повторяющиеся процессы — стандартная идиома DES. Ни одна сущность никогда не опрашивается внешним циклом. У каждой живой сущности в любой момент есть ровно одно ожидающее событие `EntityActivityEvent`; выполнение этого события реализует поведение сущности и планирует своего преемника — это самопланирование и делает процесс повторяющимся в терминах DES, и именно поэтому цепочка мёртвой сущности просто останавливается (для неё никто не планирует преемника).

Один владелец на каждую мутацию. Каждое изменение состояния имеет ровно один класс события, отвечающий за него — намеренно, чтобы на вопрос “почему X изменилось” всегда было ровно одно место для ответа:

Мутация	Владелец
Старение, расход энергии, рост, решение о движении, оценка правил	<code>EntityActivityEvent.execute()</code>
Эффекты успешного нападения хищника	<code>PredationEvent.execute()</code>
Создание потомства, статистика рождений, первое событие активности	<code>ReproductionEvent.execute()</code>
Флаг <code>alive</code> , удаление из мира, статистика смертей — для любой причины	<code>DeathEvent.execute()</code> — единственное место вызова <code>Entity.die()</code>
Запись изменения позиции	<code>MovementEvent</code> — только наблюдение, само перемещение уже произошло внутри <code>EntityActivityEvent</code>

Ленивая инвалидация. Сущность может умереть в промежутке между тем, как на неё запланировали событие, и тем, как это событие реально выполнится. У `PriorityQueue` нет эффективной операции удаления произвольного элемента, поэтому ничто активно не ищет по FEL, чтобы отменить устаревшие записи — вместо этого каждое событие в начале своего `execute()` перепроверяет `isAlive()` / `isPendingRemoval()` и молча ничего не делает, если данные устарели. `Entity.pendingRemoval` ([Entity.java:38](#)) — этот флаг, выставляется синхронно в момент, когда удаление запрашивается ([Entity.java:165](#) `scheduleRemoval`), ещё до того, как выполнится `DeathEvent`, который реально переключает `alive`.

Детерминированное разрешение ничьих. `java.util.PriorityQueue` не гарантирует порядок FIFO среди элементов, которые сравниваются как равные. `SimulationEvent.compareTo` ([SimulationEvent.java:87](#)) разрывает ничью по строго монотонному `sequenceNumber` (`AtomicLong`, присваивается при создании) — два события, запланированные на один и тот же тик, всегда обрабатываются в порядке создания.

Известное, заявленное ограничение — сказать самому, не дожидаясь вопроса. “Time is integer-tick-valued, and in this particular ecosystem model, every entity needs an activity roughly every tick, so the *visible cadence* resembles a fixed timestep. The load-bearing, architectural property isn’t that inter-event gaps happen to be large — it’s that the clock is FEL-driven (it always adopts a popped event’s own time) and that state only ever changes inside `execute()`. A model where gaps

were large (say, a plant that only acts every 50 ticks) would immediately look different from a polling loop; this one doesn't have to, to still be genuinely event-driven." **Смысл:** время выражено целочисленными тиками, и в этой конкретной модели экосистемы каждой сущности нужна активность примерно на каждом тике, поэтому *видимый ритм* напоминает фиксированный шаг. Несущее архитектурное свойство — не то, что промежутки между событиями большие, а то, что часы управляются FEL (всегда принимают собственное время извлечённого события) и что состояние меняется только внутри `execute()`. Модель, где промежутки *действительно* были бы большими (скажем, растение, действующее раз в 50 тиков), сразу выглядела бы иначе, чем цикл с опросом; этой модели для того, чтобы оставаться по-настоящему событийно-управляемой, это не требуется.

Часть C — Теория DSL правил / теория компиляторов

Конвейер, по порядку: свободный текст → **Lexer** (лексер/токенизатор) → **Parser** (парсер рекурсивного спуска, написан вручную, без ANTLR или другого генератора) → **AST** (sealed-интерфейсы + records) → статическая **валидация** относительно `RuleVocabulary` → **Evaluator** (время выполнения, один раз на сущность за тик).

Грамматика (в стиле EBNF, приоритет от низкого к высокому):

```
rule      ::= name "|" targetType "|" condition "|" actionList
condition ::= orExpr
orExpr    ::= andExpr ( "OR" andExpr )*
andExpr   ::= unary ( "AND" unary )*
unary     ::= "NOT" unary | comparison | "(" orExpr ")"
comparison ::= numExpr compareOp numExpr
numExpr   ::= term ( "+" | "-" ) term
term      ::= factor ( "*" | "/" ) factor
factor    ::= NUMBER | IDENTIFIER | "-" factor | "(" numExpr ")"
actionList ::= action ( "," action )*
action     ::= IDENTIFIER ( mutationOp numExpr )?
mutationOp ::= "=" | "+=" | "-=" | "*=" | "/="
```

Приоритет, от низкого к высокому: OR < AND < NOT < сравнение < + - < * / . Это классическая форма грамматики выражений, написанной вручную рекурсивным спуском — каждое правило грамматики становится одним методом парсера, а приоритет операций кодируется тем, *какой метод вызывает какой* (`orExpr` вызывает `andExpr`, тот вызывает `unary` ...), а не таблицей приоритетов.

Lexer. Превращает сырые символы в плоский поток токенов (`TokenType` + лексема + позиция). Ничего не знает о структуре грамматики — только классы символов (цифра, буква, символ оператора) и распознавание ключевых слов (AND , OR , NOT).

Parser. Рекурсивный спуск: каждый нетерминал грамматики выше — это один метод (`parseOrExpr()`, `parseAndExpr()`, ...), и каждый метод вызывает метод следующего, более высокого приоритета для своих операндов — именно так приоритет операций реализуется структурно, а не через таблицу.

AST — sealed-интерфейсы + records (Java 21), не классический visitor. `ConditionNode` (`ConditionNode.java:11`) — `sealed`, разрешает только `And`, `Or`, `Not`, `Compare` — все `record`. Та же форма у `NumericNode` (`NumericNode.java:18`: `Literal`, `Reference`, `BinaryOp`) и `RuleAction` (`RuleAction.java:12`: `Mutation`, `Command`). `Evaluator` обрабатывает их исчерпывающим `switch` с сопоставлением с образцом (`Evaluator.java:37`, `:53`, `:64`, `:98`) — компилятор откажется собирать проект, если пропущен хоть один `case`, а это даёт то самое “закрыто и проверено на исчерпываемость”, для чего классическому объектно-ориентированному паттерну `visitor` (двойная диспетчеризация через пару `accept()` / `visit()` на каждом узле) понадобился бы отдельный интерфейс и полная реализация `visitor`-а, причём даже тогда — без проверки исчерпываемости компилятором до появления этой возможности в Java 21.

Статическая vs. динамическая (во время выполнения) валидация — один последовательный принцип. Всё, что можно обнаружить без живого состояния симуляции — неизвестный атрибут, несовпадение типа цели/команды, буквальный ноль-делитель — это жёсткое исключение `RuleParseException` во время загрузки (`RuleVocabulary.java:101-144`, `requireReadable` / `requireWritable` / `requireCommand`). Всё, что можно обнаружить только по меняющемуся состоянию во время выполнения — устаревшая запланированная ссылка, делитель, который стал нулём только потому, что его только что обнулило другое правило, — это плавный, молчаливый по-ор внутри `Evaluator`, никогда не падение.

Срабатывание по уровню (level-triggered). Правило срабатывает заново на каждом тике, пока его условие истинно — нет флага “уже сработало” на правило, что проще, чем срабатывание по фронту (`edge-triggered` — “сработать один раз, когда условие стало истинным”), но означает, что автору правила нужно самому встраивать гистерезис в правило, если хочется поведения по фронту (например, объединить условие с атрибутом-таймаутом).

Числовая безопасность. Вся арифметика правил — `double`. Деление на динамический ноль во время выполнения даёт `NaN` / `Infinity`; `Evaluator` трактует любое из них как ложное условие или как пропущенную мутацию — никогда как значение, которое распространяется в реальное состояние сущности.

Честная граница времени компиляции — формулировать точно. Составление *новой логики правил* из уже зарегистрированных в `RuleVocabulary` атрибутов/операторов/команд не требует ни изменений Java, ни перекомпиляции. Открытие по-настоящему *нового* атрибута или команды требует одного вызова `registerReadable` / `registerWritable` / `registerCommand` и перекомпиляции. Обе половины этой фразы верны — говори обе, не только выгодную половину.

Часть D — Принципы дизайна, упомянутые в отчёте по имени

- **Принцип подстановки Лисков (Liskov Substitution Principle).** Подтип должен быть применим везде, где ожидается супертип, без неожиданного поведения. Упомянут как причина, почему `Plant` не под `Animal` (A.3), и как собственный контракт `Movable` (“вызов `move()` должен приводить к валидному изменению позиции”).

- **Принцип разделения интерфейсов (Interface Segregation Principle).** Ни один класс не должен быть вынужден реализовывать метод, для которого у него нет осмысленного тела. Упомянут как причина, *почему* `Movable` и `Reproducible` — два интерфейса, а не один (A.7) — `Plant` является `Reproducible`, но не `Movable`.
- **Принцип открытости/закрытости (Open/Closed Principle).** Открыт для расширения, закрыт для изменения. Это формальное имя для A.9 (Расширяемость).

Часть Е — Сжатый глоссарий (последние пять минут перед входом)

- **Инкапсуляция** = контроль доступа к состоянию *этого объекта* (контракты методов).
Соккрытие информации = свободна ли *реализация иерархии* меняться позже (объявления полей, 3-шаговый метод). Разные оси — не путать.
- **Полиморфизм включения** разрешается **во время выполнения**, по фактическому классу объекта (`update()`, `execute()`). **Перегрузка** разрешается **во время компиляции**, по типам аргументов в месте вызова. **Параметрический** = обобщения (generics) (`EventQueue<T extends SimulationEvent>`). **Приведение (coercion)** = неявное расширение типа, вызванное явным приведением у соседнего операнда.
- **Управляется FEL** = часы принимают собственную метку времени извлечённого события.
Фиксированный шаг = часы увеличиваются на постоянную величину, затем опрашивается всё подряд. Этот проект — первое — `advanceTo()`.
- **Проверяемое исключение** (`RuleParseException extends Exception`) = вызывающий код *обязан* поймать и восстановиться — некорректный пользовательский ввод.
Непроверяемое (`SimulationException extends RuntimeException`) = нарушение контракта программирования, обычно не восстанавливаемое.
- **Sealed-интерфейс + record + исчерпывающий switch** = закрытый AST, обрабатываемый без виртуальной диспетчеризации, с проверкой компилятором, что обработаны все случаи — осознанная альтернатива классическому паттерну visitor здесь.
- **Срабатывание по уровню (level-triggered)** = правило срабатывает заново на каждом тике, пока условие истинно (нет памяти “уже сработало”). Не по фронту (edge-triggered).